DPPO

扩散策略的强化学习原理与实现

Diffusion Policy Policy Optimization

双层 MDP、去噪概率、Critic / GAE 与 PPO 更新

Yang kunpeng

论文 arxiv.org/abs/2409.00588

项目主页 diffusion-ppo.github.io

官方代码 github.com/irom-princeton/dppo

目录

第一章 从扩散动作生成到双层 MDP **1**

1.1 Diffusion Policy 的动作表示与预训练 1

1.2 高斯概率与双层 Diffusion Policy MDP 2

第二章 从环境回报到 Diffusion PPO 更新 **5**

2.1 环境 Critic、GAE 与去噪优势分配 5

2.2 去噪 Log Probability、PPO Clipping 与梯度 7

2.3 末端去噪微调、DDIM 与噪声调度 8

2.4 完整在线训练闭环与推理过程 9

参考资料 **12**

第 1 章

从扩散动作生成到双层 MDP

1.1 Diffusion Policy 的动作表示与预训练

动作生成的两个时间尺度。机器人在环境时刻 t 接收观测 s_t，策略输出连续动作。普通高斯策略可直接写成 $\pi_\theta(a_t \mid s_t)$；Diffusion Policy（DP）则从随机噪声开始，多次调用去噪网络后才得到动作。因此，环境交互时间 t 与内部去噪时间 k 是两条不同的时间轴。在一轮去噪期间，环境观测保持不变，只有内部动作变量逐步更新。

论文用 a_t^k 表示环境时刻 t、去噪层级 k 的动作变量。$k = K$ 是最初的高斯噪声，$k = 0$ 是最终动作块：

$$a_t^K \sim \mathcal{N}(0, I), \qquad a_t^K \longrightarrow a_t^{K-1} \longrightarrow \cdots \longrightarrow a_t^0. \tag{1.1}$$

例如机械臂抓杯子，a_t^K 不是一条能够执行的控制指令；它只是待去噪的连续张量。只有生成 a_t^0 后，机器人才会按其中的控制命令运动。无论做过多少次去噪，都不应把一次去噪当成一次环境动作。

动作块、预测长度和执行长度。DPPO 沿用 Diffusion Policy 的 Action Chunk 形式。令动作维度为 d_a，预测动作块长度为 T_p，则

$$a_t^k \in \mathbb{R}^{T_p \times d_a}. \tag{1.2}$$

采样器生成 T_p 个未来动作，但实际只执行其中前 T_a 个，$T_a \le T_p$，然后重新观测环境并生成新块。论文在 MDP 记号中将实际执行的块统称为一次环境动作。只有被执行的动作会直接影响这次交互得到的奖励。官方代码的变量命名是 `horizon_steps`（预测长度）与 `act_steps`（执行长度）。

观测可以是机器人本体状态，也可以包括视觉信息。官方 PPO 模块的输入约定支持状态历史 (B, T_o, D_o) 和图像历史 (B, T_o, C, H, W)，其中 B 为批量大小，T_o 为观测历史长度。用视觉编码器或状态编码器得到条件表征后，噪声预测网络 $\epsilon_\theta(a_t^k, k, s_t)$ 接收带噪动作、扩散索引和观测条件，输出与 a_t^k 同形状的噪声预测。论

文采用过 MLP 与 UNet 作为策略网络；具体编码器与网络宽度取决于任务配置，不应把某一个实现当成方法唯一架构。

行为克隆 (BC)。先给专家动作 a^0 加噪，再让网络预测加入的噪声。标准 DDPM 前向加噪为

$$q(a^k \mid a^{k-1}) = \mathcal{N}\Big(a^k; \sqrt{\alpha_k}\, a^{k-1}, \beta_k I\Big), \tag{1.3}$$

$$a^k = \sqrt{\bar{\alpha}_k}\, a^0 + \sqrt{1-\bar{\alpha}_k}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{1.4}$$

其中 $\alpha_k = 1 - \beta_k$、$\bar{\alpha}_k = \prod_{j=1}^{k} \alpha_j$。对离线专家样本 (s, a^0)，常用的预训练目标为

$$\mathcal{L}_{\text{BC}}(\theta) = \mathbb{E}_{(s,a^0),k,\epsilon}\Big[\|\epsilon - \epsilon_\theta(a^k, k, s)\|_2^2\Big]. \tag{1.5}$$

这个目标训练的是噪声预测能力，并不直接使用环境奖励。原论文附录 BC 公式中网络的动作参数出现了 a_t^0，但紧接的说明指出输入是加噪动作；式 (1.5) 按正文机制明确标记为 a^k。

反向采样。训练好的模型在每个噪声层级给出一个高斯转移：

$$p_\theta(a^{k-1} \mid a^k, s) = \mathcal{N}\Big(a^{k-1}; \mu_k\big(a^k, \epsilon_\theta(a^k, k, s)\big), \sigma_k^2 I\Big). \tag{1.6}$$

μ_k 是由采样器确定的均值计算函数，σ_k 来自噪声调度。训练通过 ϵ_θ 的预测间接决定去噪均值。一次采样可写成

$$a^{k-1} = \mu_\theta(a^k, k, s) + \sigma_k \xi_k, \qquad \xi_k \sim \mathcal{N}(0, I). \tag{1.7}$$

这些方程不仅说明动作如何生成，也说明了为什么下一节可以对每一步随机转移求 Log Probability。

1.2 高斯概率与双层 Diffusion Policy MDP

单步概率可算，最终动作概率难算。固定当前输入 a^k、环境状态 s 和采样得到的 a^{k-1} 后，式 (1.6) 的均值、方差和实际样本全部已知；高斯分布的密度可以直接代入计算。

对于已展开成 $D = T_p d_a$ 维的动作变量，若协方差为 $\sigma_k^2 I$，则

$$\log p_\theta(a^{k-1} \mid a^k, s) = -\frac{D}{2}\log(2\pi) - D\log\sigma_k - \frac{\|a^{k-1} - \mu_\theta\|_2^2}{2\sigma_k^2}. \tag{1.8}$$

这是条件密度，描述已知 a^k 时，得到特定 a^{k-1} 的密度。连续变量的点密度不等同于离散事件概率，密度数值也不要求小于 1。

另一方面，整个路径的联合密度是

$$p_\theta(a^{0:K} \mid s) = p(a^K) \prod_{k=1}^{K} p_\theta(a^{k-1} \mid a^k, s). \tag{1.9}$$

如果只关心最终动作 a^0，必须对所有中间噪声变量积分：

$$\pi_\theta(a^0 \mid s) = \int p(a^K) \prod_{k=1}^{K} p_\theta(a^{k-1} \mid a^k, s)\, \mathrm{d}a^{1:K}. \tag{1.10}$$

以两步去噪 $a^2 \to a^1 \to a^0$ 为例，若实际路径中的 $a^1 = 0.4$，可直接计算 $p_\theta(a^0 \mid a^1 = 0.4, s)$；但若只给出 a^0，还必须汇总 a^1 取任意可能值的贡献。因为每一步均值由非线性神经网络计算，式 (1.10) 通常不能方便地解析求值。DPPO 因而保留真实采样的路径，转而优化式 (1.8) 所定义的单步密度。

外层环境 MDP。原来的机器人控制问题是 $\mathcal{M}_{\text{env}} = (\mathcal{S}, \mathcal{A}, P_0, P, R)$。在环境时刻 t，执行最终动作块 a_t^0，得到奖励 $R(s_t, a_t^0)$，环境转移至 $s_{t+1} \sim P(\cdot \mid s_t, a_t^0)$。

内层去噪 MDP。将扩散过程的一个随机转移看作一次策略决策。原论文定义扩展状态、扩展动作和扩展奖励：

$$\bar{s}_{t,k} = (s_t, a_t^{k+1}), \qquad \bar{a}_{t,k} = a_t^k, \qquad \bar{R}_{t,k} = \begin{cases} 0, & k > 0, \\ R(s_t, a_t^0), & k = 0. \end{cases} \tag{1.11}$$

严格的扩展时间可记为 $\bar{t}(t,k) = tK + (K - k - 1)$。当 $k > 0$，环境不动，只将新生成的 a_t^k 放进下一内部状态 (s_t, a_t^k)；当 $k = 0$，机器人执行 a_t^0，得到新的环境状态，并重新采样 $a_{t+1}^K \sim \mathcal{N}(0, I)$。

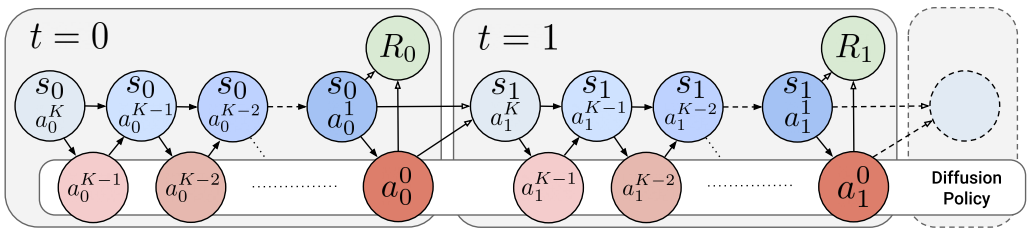


图 1.1: 双层 Diffusion Policy MDP：去噪动作在内层连续产生，只有最后一步产生环境奖励并推进到下一机器人决策状态。原论文示意图。

在扩展 MDP 中，真正的策略不是难算的最终动作边缘密度，而是

$$\bar{\pi}_\theta(\bar{a}_{t,k} \mid \bar{s}_{t,k}) = p_\theta(a_t^k \mid a_t^{k+1}, s_t) = \mathcal{N}\left(a_t^k; \mu_{\theta,t,k}, \sigma_{k+1}^2 I\right). \tag{1.12}$$

这里 $\mu_{\theta,t,k} = \mu(a_t^{k+1}, \epsilon_\theta(a_t^{k+1}, k+1, s_t))$。注意下标：扩展动作是 a_t^k，其输入为 a_t^{k+1}；当 $k = 0$ 时，扩展动作 a_t^0 才是最终执行动作。式 (1.12) 给 PPO 提供了可计算的策略概率密度。

为什么优化内部转移有用？执行奖励虽然只发生在 $k = 0$，但最早的去噪结果会影响随后每个去噪步骤，最后影响真实机器人动作。对包含整条采样路径的概率求梯度时，有

$$\nabla_\theta \log p_\theta(a_t^{0:K} \mid s_t) = \sum_{k=0}^{K-1} \nabla_\theta \log \bar{\pi}_\theta(a_t^k \mid a_t^{k+1}, s_t), \tag{1.13}$$

因为初始高斯分布 $p(a_t^K)$ 不依赖 θ。这是从联合概率乘积得到的数学恒等式。它说明对整条路径的策略梯度可以分解成每一次去噪的 Log Probability 梯度，完全不需要对环境动力学反向传播。DPPO 并不是近似计算式 (1.10)；它是改变策略优化单位，在扩展 MDP 内直接进行 Policy Gradient。

第 2 章

从环境回报到 Diffusion PPO 更新

2.1 环境 Critic、GAE 与去噪优势分配

奖励发生在环境时间，而不是去噪时间。机器人在时刻 t 生成一个动作块，并与环境交互后得到 r_t。虽然这次动作块可能由 K 次去噪构成，但中间 $K-1$ 次转移没有独立的环境奖励。DPPO 因此先按环境决策序列计算 Advantage，再分配给内部去噪步骤。

环境级 Critic 预测状态价值 $V_\phi(s_t)$，表示从当前状态出发的预计累计折扣回报。虽然扩展 MDP 的状态包括 (s_t, a_t^{k+1})，论文实际选择仅用环境状态训练 Critic，不为不同噪声水平建立独立 Value Head。这样避免让 Critic 从高维、含噪的动作中额外估计价值。

利用环境折扣 γ_{env} 计算 TD Error：

$$\delta_t = r_t + \gamma_{\text{env}} V_\phi(s_{t+1}) - V_\phi(s_t). \tag{2.1}$$

再用 GAE（Generalized Advantage Estimation）递推：

$$\hat{A}_t^{\text{env}} = \delta_t + \gamma_{\text{env}} \lambda \hat{A}_{t+1}^{\text{env}}. \tag{2.2}$$

终止状态 $V_\phi(s_T) = 0$；如果采样因长度限制被截断，则应按截断约定使用 Bootstrap。λ 控制长时间奖励估计与短时间 TD 估计的权衡。式 (2.2) 沿着机器人环境时间 t 计算，并不是沿扩散层级 k 计算。

例如一次机器人抓取产生两次环境决策：$r_0 = 0, r_1 = 1$；旧 Critic 给出 $V(s_0) = 0.4, V(s_1) = 0.8, V(s_2) = 0$。取 $\gamma_{\text{env}} = 0.9, \lambda = 0.95$，则

$$\delta_1 = 1 - 0.8 = 0.2, \tag{2.3}$$

$$\delta_0 = 0 + 0.9 \times 0.8 - 0.4 = 0.32, \tag{2.4}$$

$$\hat{A}_1^{\text{env}} = 0.2, \tag{2.5}$$

$$\hat{A}_0^{\text{env}} = 0.32 + 0.9 \times 0.95 \times 0.2 = 0.491. \tag{2.6}$$

第一个决策虽然没有立即拿到奖励，但成功的后续动作仍然能通过 GAE 为它提供正 Advantage。

去噪折扣。对于同一次环境决策 t，DPPO 将 $\hat{A}_t^{\text{env}}$ 分配给内部各步骤：

$$\hat{A}_{t,k} = \gamma_{\text{denoise}}^{k} \hat{A}_t^{\text{env}}, \quad k = K-1, \ldots, 0. \tag{2.7}$$

$0 < \gamma_{\text{denoise}} < 1$ 是独立于环境折扣的超参数。k 越大，转移越早、噪声水平越高，权重越小。这个折扣是论文设计的梯度加权方法，不等于在每个去噪步骤计算了不同的真实环境奖励。

对于示例的第一次环境决策，令 $K = 3, \gamma_{\text{denoise}} = 0.9$，式 (2.6) 给出 $\hat{A}_0^{\text{env}} = 0.491$。于是 $a^3 \to a^2$、$a^2 \to a^1$、$a^1 \to a^0$ 的权重分别是 $0.81, 0.9, 1$；三个 Advantages 约为 $0.3977, 0.4419, 0.4910$。三个数符号相同，但梯度权重不同。Critic 仍然只有一个环境级 Value 估计。

官方 `model/diffusion/diffusion_ppo.py` 将去噪索引变换为指数，并在 PPO Loss 前乘到 Advantage 上。其核心计算包括：

```
discount = torch.tensor([
    gamma_denoising ** (K_ft - i - 1)
    for i in denoising_inds
], device=advantages.device)
advantages = advantages * discount
```

此段按官方实现等价改写，i 为微调去噪步骤在采样链中的顺序索引，K_{ft} 为参与微调的去噪步数；官方类中的相应字段分别是 `gamma_denoising` 与 `ft_denoising_steps`。$i = 0$ 对应更早的可训练转移，因此指数为 $K_{\text{ft}} - 1$。

Critic 的监督目标。论文附录明确使用环境累计折扣回报作 Value 回归目标：

$$G_t^{\text{MC}} = \sum_{\ell=0}^{T-t-1} \gamma_{\text{env}}^{\ell} r_{t+\ell}, \qquad \mathcal{L}_\phi = \mathbb{E}_t\big[(V_\phi(s_t) - G_t^{\text{MC}})^2\big]. \tag{2.8}$$

这与常见 PPO 实现采用 $\hat{A}_t + V_{\text{old}}(s_t)$ 作为 Value Target 的写法不应混为一谈。DPPO 源码对 Value Loss 还允许可选的 Value Clipping；关键是 Actor 使用 Advantage，Critic 拟合 Return，两者作用不同。

2.2 去噪 Log Probability、PPO Clipping 与梯度

为什么先保存旧策略概率？ DPPO 是 On-policy PPO。每轮采样前固定 θ_{old}，用旧策略生成带有中间去噪状态的轨迹；更新时在同一个 (s_t, a_t^{k+1}, a_t^k) 上重新评估当前策略的 Log Probability：

$$\ell_{t,k}^{\text{old}} = \log p_{\theta_{\text{old}}}(a_t^k \mid a_t^{k+1}, s_t), \tag{2.9}$$

$$\ell_{t,k}(\theta) = \log p_\theta(a_t^k \mid a_t^{k+1}, s_t). \tag{2.10}$$

不可以在求当前概率时重新随机生成 a_t^k，否则分子、分母就不再对应同一次旧策略采样。

两次概率之比是

$$\rho_{t,k}(\theta) = \frac{p_\theta(a_t^k \mid a_t^{k+1}, s_t)}{p_{\theta_{\text{old}}}(a_t^k \mid a_t^{k+1}, s_t)} = \exp\big(\ell_{t,k}(\theta) - \ell_{t,k}^{\text{old}}\big). \tag{2.11}$$

$\rho = 1$ 表示当前与旧策略在该采样点的密度相同；$\rho > 1$ 表示当前策略对这个样本赋予更高密度。对于多维动作，理想化的联合高斯 Log Probability 是式 (1.8) 的各维求和；代码中还有额外的维度归约，稍后说明。

PPO Clipped Objective。对每个去噪转移定义

$$J_{t,k}(\theta) = \min\Big(\rho_{t,k}\hat{A}_{t,k},\ \text{clip}(\rho_{t,k}, 1-\epsilon_k, 1+\epsilon_k)\hat{A}_{t,k}\Big). \tag{2.12}$$

Actor 最大化所有样本的平均 $J_{t,k}$，训练时等价于最小化 $\mathcal{L}_{\text{actor}} = -\mathbb{E}[J_{t,k}]$。当 Advantage 为正，策略倾向于增加该去噪转移的密度；当 Advantage 为负，策略倾向于减少密度。ϵ_k 限制继续推动比值所带来的目标增益，不是一个绝对禁止参数变化的硬约束。

例如某步有 $\hat{A} = 0.6$。若 $\rho = 1.1, \epsilon = 0.2$，则 $J = 0.66$；若 $\rho = 1.5$，未裁剪项是 0.90，但上侧裁剪项是 $1.2 \times 0.6 = 0.72$，所以 $J = 0.72$。正 Advantage 样本超过上侧裁剪阈值之后，这一项不再鼓励密度继续增加。负 Advantage 的下侧裁剪同理限制进一步压低密度的激励。

PPO Loss 如何进入噪声预测网络？ 对于一个未被裁剪且 Advantage 为正的样本，$\mathcal{L} = -\hat{A}\rho$，于是

$$\nabla_\theta \mathcal{L} = -\hat{A}\rho\, \nabla_\theta \log p_\theta(a^k \mid a^{k+1}, s), \tag{2.13}$$

$$\nabla_\theta \log p_\theta = \frac{(a^k - \mu_\theta)^\top}{\sigma^2} \nabla_\theta \mu_\theta. \tag{2.14}$$

第二式假设该步高斯方差固定且为各向同性。由于 $\mu_\theta = \mu(a^{k+1}, \epsilon_\theta(a^{k+1}, k+1, s))$，反向传播路径为

$$\mathcal{L}_{\text{PPO}} \longrightarrow \log p_\theta \longrightarrow \mu_\theta \longrightarrow \epsilon_\theta \longrightarrow \theta. \tag{2.15}$$

正 Advantage 会倾向于将去噪均值推向已采样的结果；负 Advantage 则倾向于使密度降低。实际优化由整批样本及 Clipping 共同决定，不能把单条轨迹理解为一个确定的网络均值更新量。

与官方代码的对应及差异。官方 `PPODiffusion.loss` 的核心 PPO 片段对应如下形式：

```
logratio = newlogprobs - oldlogprobs
ratio = logratio.exp()
loss_a = -advantages * ratio
loss_b = -advantages * ratio.clamp(1-eps, 1+eps)
actor_loss = torch.maximum(loss_a, loss_b).mean()
```

这是按官方代码等价重写的紧凑表达；真实实现中的 ϵ_k 是可以随去噪步骤变化的张量。此外源码在构造 Ratio 前，对每个坐标的 Log Probability 先裁剪至 $[-5, 2]$，再沿动作时间和动作维度取均值。因此代码中的 *Ratio* 是裁剪后、按动作维度平均 *Log Probability* 的指数差，与式 (2.11) 所表示的完整联合高斯密度比值并不严格相同。这一数值稳定化和归一化选择可能改变实际更新幅度，不能在复现时忽略。

另一个实现细节是 `reward_horizon`：官方 PPO Loss 对预测动作块只保留需要接受训练信号的前部动作坐标，再做概率归约。代码对 ϵ_k 采用随去噪步骤变化的指数插值；详细实现应以配置及源码为准，不应把任意固定的 $\epsilon = 0.2$ 当作论文统一默认值。

2.3 末端去噪微调、DDIM 与噪声调度

只训练最后 K' 步去噪。一次 DDPM 动作生成可能需要 $K = 100$ 步。若所有步骤都参与 PPO，就需存储、重算并更新大量去噪转移。DPPO 可复制一份 BC 预训练的去噪网络：原始网络 θ 冻结，前 $K - K'$ 步继续使用它；参数 θ_{ft} 只负责并更新最后 K' 步：

$$a^K \xrightarrow[\text{冻结 } \theta]{K-K' \text{ 步}} a^{K'} \xrightarrow[\text{训练 } \theta_{\text{ft}}]{K' \text{ 步}} a^0. \tag{2.16}$$

这些是去噪步数，与动作块包含几个机器人控制命令不是同一个概念。即使前面的网络冻结，它输出的中间动作仍然决定后面可训练网络的输入。

DDIM 减少采样步数。论文也允许在 RL 微调时使用更短的 DDIM 采样链。原文给出的随机转移写作

$$p_\theta^{\mathrm{DDIM}}(a^{k-1} \mid a^k, s) = \mathcal{N}\left(a^{k-1}; \mu_\theta^{\mathrm{DDIM}}, \eta\sigma_k^2 I\right), \tag{2.17}$$

其中 η 控制随机程度。这是原论文使用的方差记号；实际采样公式及不同实现对 η、标准差的参数化需对照采样器代码。论文在有关实验中采用训练 $\eta = 1$ 保留随机探索、评估 $\eta = 0$ 进行确定性生成。$\eta = 0$ 时分布退化为确定性转移，不能用普通非退化高斯密度直接照搬训练 Log Probability 公式。

探索噪声与概率估计噪声。DDPM 标准调度在最后几步可能将噪声降得极小，而强化学习需要足够随机性探索新动作。DPPO 在采样时设置更高的噪声下限 $\sigma_{\min}^{\mathrm{exp}}$，在计算去噪 Log Probability 时还设置 $\sigma_{\min}^{\mathrm{prob}}$ 以抑制数值过大的 Log Probability。论文给出的示例分别约为 $0.01 \sim 0.1$ 与 0.1，不是所有任务的固定配置。两个标准差下限的用途不同：前者影响采样行为，后者影响训练中评估的概率值；若二者不一致，计算 Ratio 时用的分布就不应被称为行为采样分布的严格精确密度。

MLP 与 UNet 对 Action Chunk 的影响。论文采用 MLP 与 UNet 两类去噪头。两者均以观测为条件输出动作噪声。由于 MLP 的全连接结构对整块动作耦合较强，论文报告在 $T_a < T_p$ 的设置下存在不稳定现象；UNet 对时间动作维的处理更灵活，因而可以用较长的预测块获得上下文，再只执行较短动作块。论文讨论过 $T_p = 16, T_a = 8$ 的例子；这些值用于解释设计，不是 DPPO 的统一固定维度。

2.4 完整在线训练闭环与推理过程

Rollout Buffer 的数据内容。每一次环境决策 t 必须保留 s_t、最终执行动作块、环境奖励 r_t、终止标志、旧 Critic 值，以及参与训练的去噪路径 $\{a_t^k\}$ 与对应旧 Log Probability。对可训练的每一步 k，可组织成一条训练记录

$$(s_t,\ a_t^{k+1},\ a_t^k,\ k,\ \ell_{t,k}^{\mathrm{old}},\ \hat{A}_t^{\mathrm{env}},\ G_t). \tag{2.18}$$

一个环境时间步对应多个去噪记录，但这些记录共享同一份环境奖励和环境价值估计。独立采样的新环境轨迹用于下一轮 PPO 更新；同一轮采样数据可在固定旧策略下进行多轮 Minibatch 更新。

训练循环。论文附录 Algorithm 1 包括 BC 预训练、初始化 Critic、并行环境采样、计算 GAE、按去噪步骤计算折扣 Advantage，以及对 Actor 与 Critic 的交替更新。下列紧凑伪代码保留相同数据流；它仅表示算法次序而不是可直接执行的训练程序：

```
pretrain_diffusion_bc(expert_dataset)
critic = initialize_value_network()
for iteration in training_iterations:
    old_policy = snapshot(policy)
    trajectories = rollout_with_chains(old_policy, envs)
    env_adv, returns = compute_gae(trajectories, critic)
    for batch in ppo_minibatches(trajectories):
        k = batch.denoising_step
        adv = env_adv[batch.env_index] * gamma_den ** k
        new_logp = policy.logp_step(batch)
        ratio = (new_logp - batch.old_logp).exp()
        update_actor_ppo(ratio, adv, batch)
        update_critic(critic, returns, batch)
```

此段为算法级伪代码；其中 update_actor_ppo 必须计算式 (2.12) 的裁剪目标，实际官方实现包含 Log Probability 归约、可选的 Advantage 标准化与分位数截断、步相关裁剪阈值、Value Clipping 和训练监控等处理。若只训练最后 K' 步，Buffer/Minibatch 只需为这些可训练转移计算 Actor Loss。

可检验的张量对应关系。官方 `PPODiffusion.loss` 接收的主要字段包含 `obs`、`chains_prev`、`chains_next`、`denoising_inds`、`returns`、`oldvalues`、`advantages` 和 `oldlogprobs`。对应的路径张量约定为 $(B, K+1, T_a, d_a)$；对去噪步骤索引、动作时间位置和动作维度的归约要逐项核对。`VPGDiffusion` 负责逐步生成动作及评估去噪概率，`PPODiffusion` 在此基础上构造 PPO Loss，`train_ppo_diffusion_agent.py` 组织环境交互和优化轮次。

一次手算。假设某次环境决策得到 $\hat{A}_t^{\text{env}} = 0.6$，两步去噪对应 $k = 1, 0$，且 $\gamma_{\text{denoise}} = 0.9$。则两个 Advantage 是 $0.54, 0.6$。设两个去噪转移的 Log Probability 差分别为 $\log(1.1), \log(1.5)$，等价于 Ratio 为 $1.1, 1.5$。若教学示例取统一裁剪阈值 $\epsilon = 0.2$，则两个 PPO 项分别为

$$J_{t,1} = \min(1.1 \times 0.54, 1.1 \times 0.54) = 0.594, \tag{2.19}$$

$$J_{t,0} = \min(1.5 \times 0.6, 1.2 \times 0.6) = 0.72, \tag{2.20}$$

$$J_t = (0.594 + 0.72)/2 = 0.657. \tag{2.21}$$

更新时最小化 $-J_t$；同时 Critic 根据环境回报进行价值回归。这个算例刻意只演示两个 *PPO* 项如何计算；真实训练中的 Ratio 还受到前述维度平均、Log Probability 裁剪与变步长 ϵ_k 等实现选择的影响。

推理与训练的分界。训练期间需执行带探索的去噪采样、保存去噪链、评估旧新概率、计算 GAE 并优化 Actor/Critic。部署时只需输入当前机器人观测，由训练好的策略产生 a_t^0，执行前 T_a 个动作，再重新观测和规划；Critic、Advantage、旧策略概率及 PPO Loss 都不参与动作生成。如果采用确定性 DDIM 评估，还会关闭随机采样噪声；是否使用 DDIM 及去噪步数，应匹配对应的训练和模型配置。

DPPO 的策略优化单位是随机去噪转移，而不是离散动作 Token，也不是动作块中的单个关节坐标。机器人任务反馈通过环境级 GAE 产生一次决策的 Advantage，经去噪折扣作用到多个高斯转移，最后由 PPO 的概率比值和裁剪目标更新噪声预测网络。这条数据流同时解释了论文的理论形式及官方实现中最关键的张量和数值处理。

参考资料

1. Ren, A. Z. et al. *Diffusion Policy Policy Optimization.* ICLR 2025. https://arxiv.org/abs/2409.00588
2. Ren et al. *DPPO official implementation.* GitHub: irom-princeton/dppo
3. Chi, C. et al. *Diffusion Policy: Visuomotor Policy Learning via Action Diffusion.* RSS 2023.
4. Schulman, J. et al. *Proximal Policy Optimization Algorithms.* 2017.
5. Schulman, J. et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation.* 2015.
6. Ho, J. et al. *Denoising Diffusion Probabilistic Models.* 2020.
7. Song, J. et al. *Denoising Diffusion Implicit Models.* 2021.